

EDS Lab Assignments

Practical 2

Name: Swamini Bhagwat

PRN: 202501100023

2.1.1 List Operations

Problem Statement:

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:

1. Add
2. Remove
3. Display
4. Quit

The program should repeatedly prompt the user to enter a choice from the menu.

Depending on the choice selected, the program should perform the following actions:

- **Add:** Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
- **Remove:** Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
- **Display:** Displays the current list of integers. If the list is empty, display "List is empty".
- **Quit:** Exits the program.
- The program should handle invalid menu choices by displaying "Invalid choice". Ensure that the program continues to prompt the user until they choose to quit (option 4).

Code:

```
l1 = []
while True:
    print("1. Add")
    print("2. Remove")
    print("3. Display")
    print("4. Quit")

    n=int(input("Enter choice: "))
    if n==1:
        add = int(input("Integer: "))
        l1.append(add)
```

```

        print(f"List after adding: {l1}")
    elif n==2:
        if len(l1)==0:
            print("List is empty")
        elif len(l1)!=0:
            remove=int(input("Integer: "))
            if remove not in l1:
                print("Element not found")
            else:
                l1.remove(remove)
            print(f"List after removing: {l1}")
    elif n==3:
        if len(l1)==0:
            print("List is empty")
        else:
            print(l1)
    elif n==4:
        break
    else:
        print("Invalid choice")

```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the file explorer shows '2.1.1.List operations'. The main editor displays the Python code for a menu-driven program to manage a list of integers. The code includes options to Add, Remove, Display, and Quit, with appropriate error handling for invalid inputs and menu choices.

Below the code editor, the 'Test cases' tab is active, showing the results of running the program against a set of test cases. The test cases are designed to verify the program's behavior for various inputs and menu choices. The results show that all test cases passed, with the program correctly handling edge cases like an empty list and invalid menu choices.

Test Case Results:

Test Case	Expected Output	Actual Output
Test case 1	1. Add 2. Remove 3. Display 4. Quit enter choice: 1 Integer: 10 List after adding: [10]	1. Add 2. Remove 3. Display 4. Quit enter choice: 1 Integer: 10 List after adding: [10]
Test case 2	1. Add 2. Remove 3. Display 4. Quit enter choice: 2 Integer: 10 Element not found	1. Add 2. Remove 3. Display 4. Quit enter choice: 2 Integer: 10 Element not found
Test case 3	1. Add 2. Remove 3. Display 4. Quit enter choice: 3 Integer: 10 List after adding: [10, 20]	1. Add 2. Remove 3. Display 4. Quit enter choice: 3 Integer: 10 List after adding: [10, 20]
Test case 4	1. Add 2. Remove 3. Display 4. Quit enter choice: 4 Integer: 10 List after adding: [10, 20]	1. Add 2. Remove 3. Display 4. Quit enter choice: 4 Integer: 10 List after adding: [10, 20]

2.1.2 Dictionary Operations

Problem Statement:

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

Code:

```
# Initial dictionary with 10 predefined records
```

```
student = {  
    1: "Amit",  
    2: "Riya",  
    3: "Kiran",  
    4: "Neha",  
    5: "Arjun",  
    6: "Pooja",  
    7: "Rahul",  
    8: "Sneha",  
    9: "Vikram",  
    10: "Anjali"  
}
```

```
# write your code here..
```

```
print("Original Dictionary:", student)
```

```

new_key = int(input())
new_value = input()
student[new_key] = new_value
print("After Insertion:", student)

update_key = int(input())
new_val = input()
if update_key in student:
    student[update_key] = new_val
print("After Update:", student)

del_key = int(input())
if del_key in student:
    del student[del_key]
print("After Deletion:", student)

print("Traversing Dictionary:")
for key, value in student.items():
    print(f"{key} : {value}")

```

Output:

The screenshot shows the CodeTANTRA IDE interface. On the left, there's a sidebar with '2.3.2. Dictionary Operations' and a list of operations to be performed: Insertion, Update, Deletion, and Traversal. The main editor displays a Python script. The first part of the script defines an initial dictionary with 10 predefined records: 'Amit', 'Riya', 'Kiran', 'Neha', 'Arjun', 'Pooja', 'Rahul', 'Sneha', 'Vikram', and 'Anjali'. The code then prompts the user for a new key and value, and inserts them into the dictionary. The output shows the dictionary after insertion.

```

1 # Initial dictionary with 10 predefined records
2 student = {}
3
4 # 1: "Amit",
5 # 2: "Riya",
6 # 3: "Kiran",
7 # 4: "Neha",
8 # 5: "Arjun",
9 # 6: "Pooja",
10 # 7: "Rahul",
11 # 8: "Sneha",
12 # 9: "Vikram",
13 # 10: "Anjali"
14
15 # write your code here...
16 print("Original Dictionary:", student)
17 key_i = int(input())
18 value_i = input()
19 student[key_i] = value_i
20 print("After Insertion:", student)
21 key_u = int(input())
22 value_u = input()
23 if key_u in student:
24     student[key_u] = value_u
25 print("After Update:", student)
26
27 key_d = int(input())
28 if key_d in student:
29     del student[key_d]
30 print("After Deletion:", student)
31
32 print("Traversing Dictionary:")
33 for k, v in student.items():
34     print(k, ":", v)

```

The screenshot shows the CodeTANTRA IDE interface after the program has executed. The top status bar indicates that 2 out of 2 shown test cases passed and 2 out of 2 hidden test cases passed. The main editor displays the output of the program, which shows the dictionary after insertion, update, and deletion, followed by the traversal of the dictionary. The output is as follows:

```

Original Dictionary: {'Amit': 1, 'Riya': 2, 'Kiran': 3, 'Neha': 4, 'Arjun': 5, 'Pooja': 6, 'Rahul': 7, 'Sneha': 8, 'Vikram': 9, 'Anjali': 10}
After Insertion: {'Amit': 1, 'Riya': 2, 'Kiran': 3, 'Neha': 4, 'Arjun': 5, 'Pooja': 6, 'Rahul': 7, 'Sneha': 8, 'Vikram': 9, 'Anjali': 10, 'Suresh': 11}
After Update: {'Amit': 1, 'Riya': 2, 'Kiran': 3, 'Neha': 4, 'Arjun': 5, 'Pooja': 6, 'Rahul': 7, 'Sneha': 8, 'Vikram': 9, 'Anjali': 10, 'Suresh': 11}
After Deletion: {'Amit': 1, 'Riya': 2, 'Kiran': 3, 'Neha': 4, 'Arjun': 5, 'Pooja': 6, 'Rahul': 7, 'Sneha': 8, 'Vikram': 9, 'Anjali': 10, 'Suresh': 11}
Traversing Dictionary:
1 : Amit
2 : Riya
3 : Kiran
4 : Neha
5 : Arjun
6 : Pooja
7 : Rahul
8 : Sneha
9 : Vikram
10 : Anjali
11 : Suresh

```

2.2.1 Linear Search Technique

Problem Statement:

Write a program to check whether the given element is present or not in the array of elements using linear search.

Input format:

- The first line of input contains the array of integers which are separated by space
- The last line of input contains the key element to be searched

Output format:

- If the element is found, print the index.
- If the element is not found, print **Not found**.

Code:

```
def linear_search(arr, key):  
    for i in range(len(arr)):  
        if arr[i] == key:  
            return i  
    return -1
```

```
arr = list(map(int,input().split()))
```

```
key = int(input())
```

```
result = linear_search(arr, key)
```

```
if result != -1:
```

```
    print(result)
```

```
else:
```

```
    print("Not found")
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the problem statement for '2.2.1. Linear search Technique' is shown, including the input/output format and a sample test case. The main editor shows the Python code for the linear search algorithm. The right sidebar displays the test results, indicating that 2 out of 2 shown test cases passed. The test cases are as follows:

Test Case	Expected Output	Actual Output
Test case 1	1 2 3 4 5 6 2	1 2 3 4 5 6 2
Test case 2	1 2 3 4 5 6 7 8 9 10 20	1 2 3 4 5 6 7 8 9 10 Not found

2.2.2 Captain of the Team

Problem Statement:

You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

Input Format:

The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

Output Format

The output should be the height (in centimeters) of the tallest player.

Code:

```
heights = list(map(int,input().split()))
tallest_player = max(heights)
print(tallest_player)
```

Output:

The screenshot displays a coding platform interface for a problem titled "2.2.2. Captain of the Team". The problem statement is: "You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team." The input format is "The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space." The output format is "The output should be the height (in centimeters) of the tallest player." The code editor shows a Python solution:

```
heights=list(map(int,input().split()))
tallest_player=max(heights)
print(tallest_player)
```

 The test cases section shows "1 out of 1 shown test case(s) passed" and "2 out of 2 hidden test case(s) passed". The test case details show the expected output as 191 and the actual output as 191.